Chapter 6. Synchronization Tools

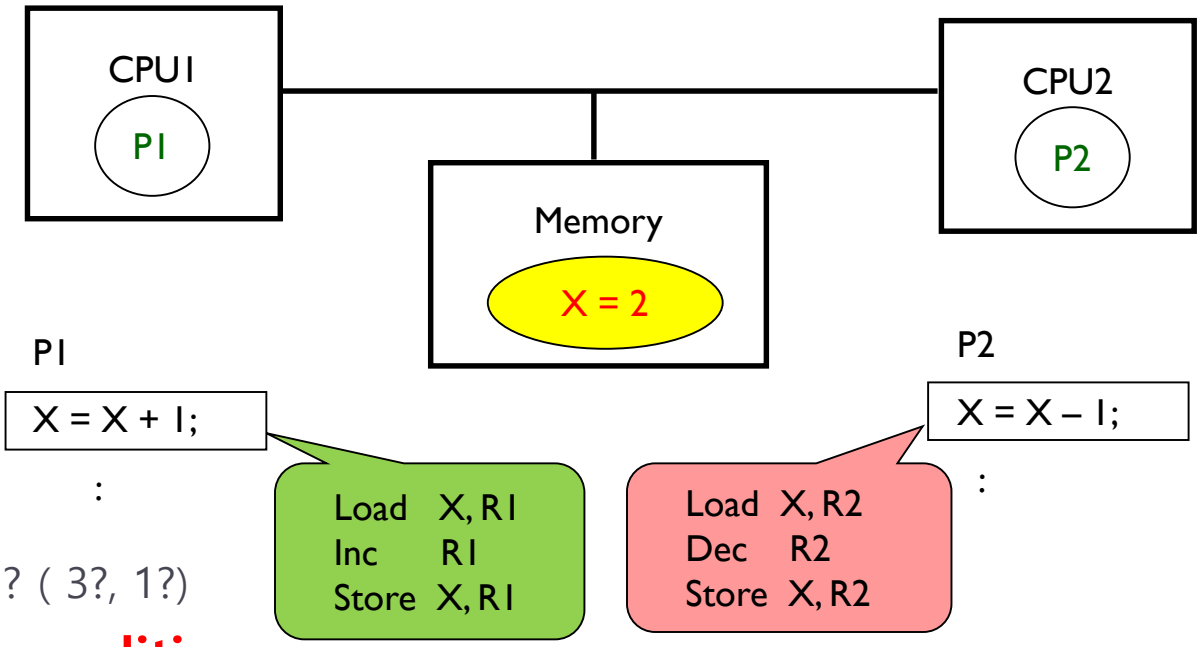
학습 목표 및 학습 내용

- **학습 목표**
  - 동기화의 필요성을 설명할 수 있다.
  - 임계 구역 문제를 설명할 수 있다.
  - 세마포어의 정의와 사용법을 설명할 수 있다.

- **학습 내용**
  - Critical Section Problem
  - Synchronization Hardware
  - Semaphore

Race Condition

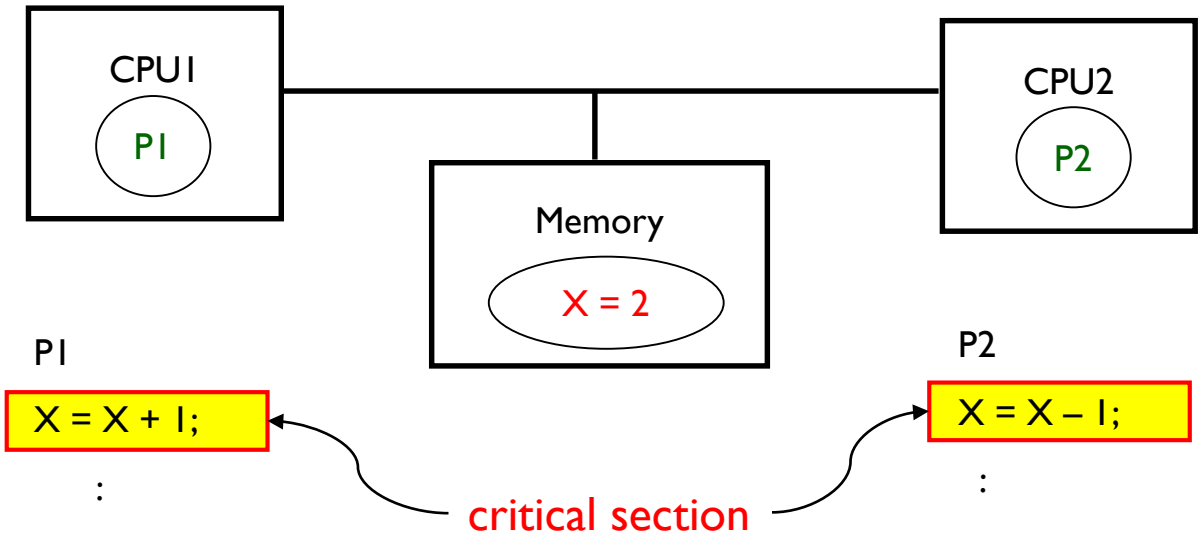
- **Concurrent access to shared data may result in data inconsistency.**



  - x=2? (3?, 1?)
- **Race condition**
  - Situation where several processes access & manipulate shared data concurrently.
  - To prevent race conditions, concurrent processes must be synchronized.
  - In uniprocessor environment, race condition can also occur by CPU scheduler.

Critical Section Problem

- **Critical section**
  - A code segment in which the shared data is accessed.



- **Critical section problem**
  - ensure that when one process is executing in its critical section, no other processes are allowed to execute in its critical section.

Critical Section Problem

- **It is required to design a protocol that the processes can use to cooperate.**

- **General structure of a typical process Pi**

```
do {
        entry section
        critical section
        exit section
        remainder section
} while (TRUE);
```

  - Entry section
    - code segment of requesting permission to enter the critical section.
  - Exit section
    - code segment of notifying the exit of the critical section.

Requisites for solution

- **Mutual Exclusion**
  - If process Pi is executing in its critical section,
  - then no other processes can be executing in their critical sections.
- **Progress**
  - If no process is executing in its critical section and
  - there exist some processes that wish to enter their critical section,
  - then the selection of the processes that will enter the critical section next cannot be postponed indefinitely.
- **Bounded Waiting**
  - A time bound must exist to prevent the starvation.

Critical-Section Handling

- Two approaches depending on if kernel is **preemptive or non-preemptive**
  - **Preemptive** – allows preemption of process when running in kernel mode
  - **Non-preemptive** – runs until exits kernel mode, blocks, or voluntarily yields CPU
    - Essentially free of race conditions in kernel mode

Peterson's Solution

- **Solution for two processes synchronization.**

- **The two processes share two variables.**
  - int turn;
    - whose turn it is to enter the critical section.

  - boolean flag[2];
    - indicates if a process is ready to enter the critical section.
    - flag[i] = true, if process P_i is ready.

Peterson's Solution

flag[j] means "j want to enter".
'turn == j' means "This is j's turn".
➔ No operation.

```
while (true) {
Entry section   flag[i] = TRUE;
                turn = j;
                while ( flag[j] && turn == j);
                critical section

Exit section    flag[i] = FALSE;

                remainder section
}
```

Algorithm for process P_i

```
while (true) {
Entry section   flag[j] = TRUE;
                turn = i;
                while ( flag[i] && turn == i);
                critical section

Exit section    flag[j] = FALSE;

                remainder section
}
```

Algorithm for process P_j

Synchronization Hardware

- **Simple tool - lock**

```
do {
        acquire lock
        critical section
        release lock
        remainder section
} while (TRUE);
```

- **Modern machines provide special atomic hardware instructions.**
  - Atomic = non-interruptable
  - Either test memory word and set value, or
  - swap contents of two memory words.

Synchronization Hardware

- **TestAndSet instruction**
  - Test and modify the content of a word atomically.

```
boolean TestAndSet (boolean *target)
{
        boolean rv = *target;
        *target = TRUE;
        return rv:
}
```

1. Return original value.
2. Set the value to true.

- **Shared boolean variable lock is initialized to false.**
- **Solution**

```
while (true) {
        while (TestAndSet(&lock )) ;   /* do nothing */
        critical section
        lock = FALSE;
        remainder section
}
```

1. Return the value of 'lock'.
2. Set 'lock' to true.

Synchronization Hardware

- **Swap instruction**
  - Switch the values of two words.

```
void Swap (boolean *a, boolean *b)
{
        boolean temp = *a;
        *a = *b;
        *b = temp:
}
```

- **Shared boolean variable lock is initialized to FALSE.**
- **Each process has a local boolean variable key.**

```
while (true) {
        key = TRUE;
        while (key == TRUE)
                swap (&lock, &key);
        critical section
        lock = FALSE;
        remainder section
}
```

"key == true" means "I want to enter".
"lock == true" means "He entered".

Semaphore

- **Semaphore S**
  - just integer variable

- **Two standard operations to modify S.**
  - wait() and signal()
    - originally called P() and V().

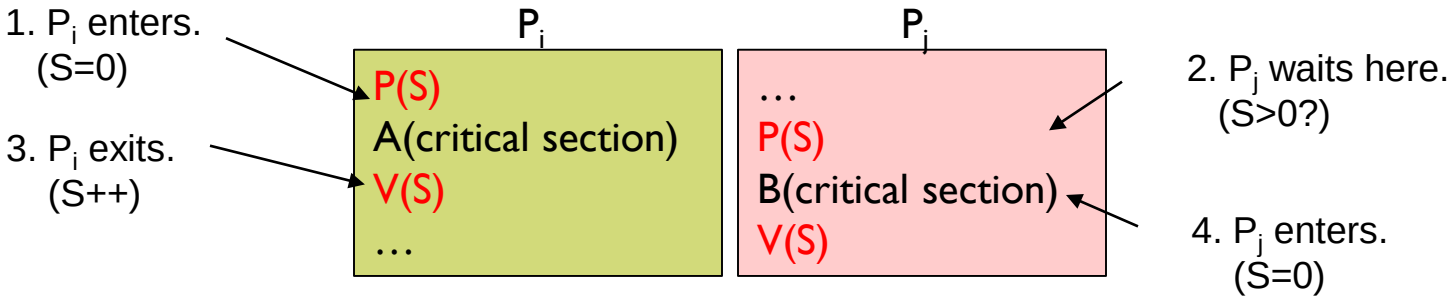
```
P(S) {
        while (S <= 0)
                ;          // noop
        S--;
}

V(S) {
        S++;
}
```

If positive, decrement & enter.
Otherwise, wait until it gets positive.

Semaphore

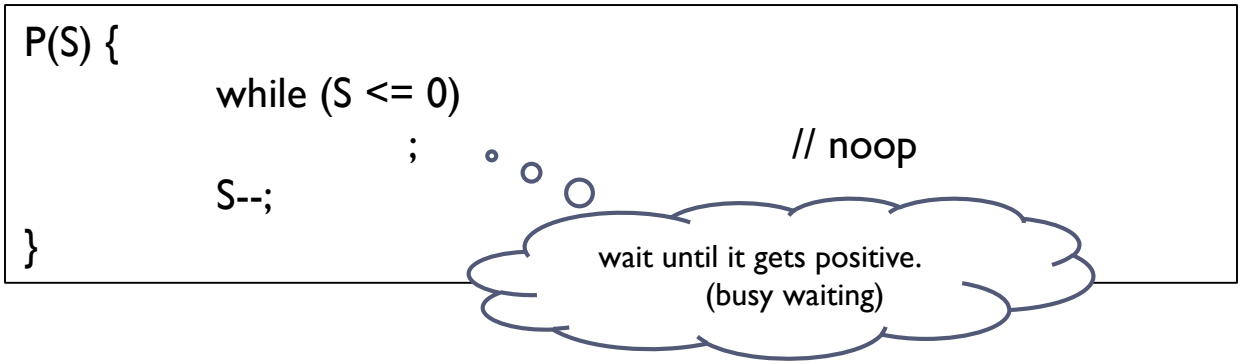
- **Usage of semaphore**
  - There are two processes P_i & P_j and two critical sections A & B.
  - Initialize S to 1.



- **Binary semaphore**
  - integer value can range only between 0 and 1.
  - Provides mutual exclusion
- **Counting semaphore**
  - integer value can range over an unrestricted domain.

Semaphore implementation

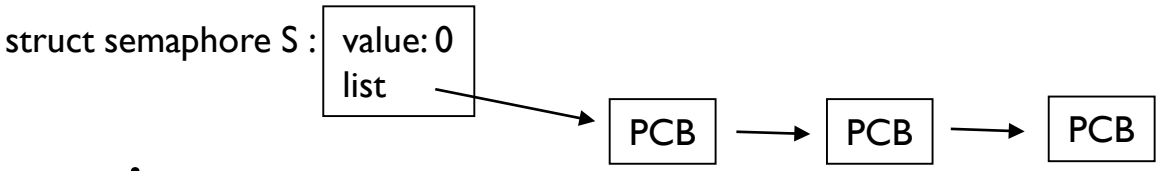
- **Busy waiting**
  - While a process is in critical section, other processes must loop continuously.
  - is also called a spinlock.
  - wastes CPU cycles.



Semaphore implementation

- **Block and wakeup**
  - There is an associated waiting queue for each semaphore.

```
Typedef struct {
        int value;                  // semaphore
        struct process *list;       // waiting queue
} semaphore;
```



- **Two operations**
  - Block
    - Suspends the process that invoked P(S) itself.
    - Inserts this process' PCB into wait queue for semaphore S.
  - Wakeup(P)
    - Removes a process P's PCB from wait queue for semaphore S.
    - Resumes the execution of P.

Semaphore implementation

- **Implementation of semaphore with block and wakeup.**

```
P(S){
        S.value--;
        if (S.value < 0) {
                add this process to waiting queue.
                block();
        }
}

V(S){
        S.value++;
        if (S.value <= 0) {
                remove a process P from the waiting queue.
                wakeup(P);
        }
}
```

Semaphore implementation

- **Which is better?**
  - Busy-waiting
    - No context switching is required.
    - It is good when the length of critical section is short.
  - Block-wakeup
    - Context switching is required.
    - It is good when the length of critical section is long.

Summary

- **Race condition is a situation where several processes access and manipulate shared data concurrently.**
- **The critical section is a code segment in which the shared data is accessed.**
- **A semaphore S is an integer variable that is accessed only through P() and V() operations.**
- **Busy waiting is good for short critical section, while block-wakeup is good for long critical section.**